In [1]:
```
## 1.IMPORT
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, classification_report
)

sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)
pd.set_option("display.max_columns", 100)
```

In [3]:
```
## 2.LOAD DATA

DATA_PATH = r"C:\Users\HP\Downloads\WA_Fn-UseC_-HR-Employee-Attrition.csv
df = pd.read_csv(DATA_PATH)
print("Shape:", df.shape)
df.head(10)
```

```
Shape: (1470, 35)
```

Out[3]:

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Ed
0	41	Yes	Travel_Rarely	1102	Sales	1	
1	49	No	Travel_Frequently	279	Research & Development	8	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	
4	27	No	Travel_Rarely	591	Research & Development	2	
5	32	No	Travel_Frequently	1005	Research & Development	2	
6	59	No	Travel_Rarely	1324	Research & Development	3	
7	30	No	Travel_Rarely	1358	Research & Development	24	
8	38	No	Travel_Frequently	216	Research & Development	23	
9	36	No	Travel_Rarely	1299	Research & Development	27	

In [4]:
```
## 3.OVERVIEW

df.info()
df.describe().T
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
 #   Column                    Non-Null Count  Dtype 
---  ------                    --------------  ----- 
 0   Age                       1470 non-null   int64 
 1   Attrition                 1470 non-null   object
 2   BusinessTravel            1470 non-null   object
 3   DailyRate                 1470 non-null   int64 
 4   Department                1470 non-null   object
 5   DistanceFromHome          1470 non-null   int64 
 6   Education                 1470 non-null   int64 
 7   EducationField            1470 non-null   object
 8   EmployeeCount             1470 non-null   int64 
 9   EmployeeNumber            1470 non-null   int64 
 10  EnvironmentSatisfaction   1470 non-null   int64 
 11  Gender                    1470 non-null   object
 12  HourlyRate                1470 non-null   int64 
 13  JobInvolvement            1470 non-null   int64 
 14  JobLevel                  1470 non-null   int64 
 15  JobRole                   1470 non-null   object
 16  JobSatisfaction           1470 non-null   int64 
 17  MaritalStatus             1470 non-null   object
 18  MonthlyIncome             1470 non-null   int64 
 19  MonthlyRate               1470 non-null   int64 
 20  NumCompaniesWorked        1470 non-null   int64 
 21  Over18                    1470 non-null   object
 22  OverTime                  1470 non-null   object
 23  PercentSalaryHike         1470 non-null   int64 
 24  PerformanceRating         1470 non-null   int64 
 25  RelationshipSatisfaction  1470 non-null   int64 
 26  StandardHours             1470 non-null   int64 
 27  StockOptionLevel          1470 non-null   int64 
 28  TotalWorkingYears         1470 non-null   int64 
 29  TrainingTimesLastYear     1470 non-null   int64 
 30  WorkLifeBalance           1470 non-null   int64 
 31  YearsAtCompany            1470 non-null   int64 
 32  YearsInCurrentRole        1470 non-null   int64 
 33  YearsSinceLastPromotion   1470 non-null   int64 
 34  YearsWithCurrManager      1470 non-null   int64 
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
```

Out[4]:

	count	mean	std	min	25%	50
Age	1470.0	36.923810	9.135373	18.0	30.00	36
DailyRate	1470.0	802.485714	403.509100	102.0	465.00	802
DistanceFromHome	1470.0	9.192517	8.106864	1.0	2.00	7
Education	1470.0	2.912925	1.024165	1.0	2.00	3
EmployeeCount	1470.0	1.000000	0.000000	1.0	1.00	1
EmployeeNumber	1470.0	1024.865306	602.024335	1.0	491.25	1020
EnvironmentSatisfaction	1470.0	2.721769	1.093082	1.0	2.00	3
HourlyRate	1470.0	65.891156	20.329428	30.0	48.00	66
JobInvolvement	1470.0	2.729932	0.711561	1.0	2.00	3
JobLevel	1470.0	2.063946	1.106940	1.0	1.00	2
JobSatisfaction	1470.0	2.728571	1.102846	1.0	2.00	3
MonthlyIncome	1470.0	6502.931293	4707.956783	1009.0	2911.00	4919
MonthlyRate	1470.0	14313.103401	7117.786044	2094.0	8047.00	14235
NumCompaniesWorked	1470.0	2.693197	2.498009	0.0	1.00	2
PercentSalaryHike	1470.0	15.209524	3.659938	11.0	12.00	14
PerformanceRating	1470.0	3.153741	0.360824	3.0	3.00	3
RelationshipSatisfaction	1470.0	2.712245	1.081209	1.0	2.00	3
StandardHours	1470.0	80.000000	0.000000	80.0	80.00	80
StockOptionLevel	1470.0	0.793878	0.852077	0.0	0.00	1
TotalWorkingYears	1470.0	11.279592	7.780782	0.0	6.00	10
TrainingTimesLastYear	1470.0	2.799320	1.289271	0.0	2.00	3
WorkLifeBalance	1470.0	2.761224	0.706476	1.0	2.00	3
YearsAtCompany	1470.0	7.008163	6.126525	0.0	3.00	5
YearsInCurrentRole	1470.0	4.229252	3.623137	0.0	2.00	3
YearsSinceLastPromotion	1470.0	2.187755	3.222430	0.0	0.00	1
YearsWithCurrManager	1470.0	4.123129	3.568136	0.0	2.00	3

In [5]:
```
## 4.MISSING VALUE & DUPLICATES

missing = df.isnull().sum()
missing = missing[missing > 0]
print("Missing values:\n", missing if len(missing) else "None found")
print("Duplicate rows:", df.duplicated().sum())
```

```
Missing values:
 None found
Duplicate rows: 0
```

In [6]:
```python
## 5.DROP NON-INFORMATION COLUMNS

drop_cols = [c for c in ["EmployeeCount", "Over18", "StandardHours", "Emp
df = df.drop(columns=drop_cols)
print("Dropped:", drop_cols, "| New shape:", df.shape)
```

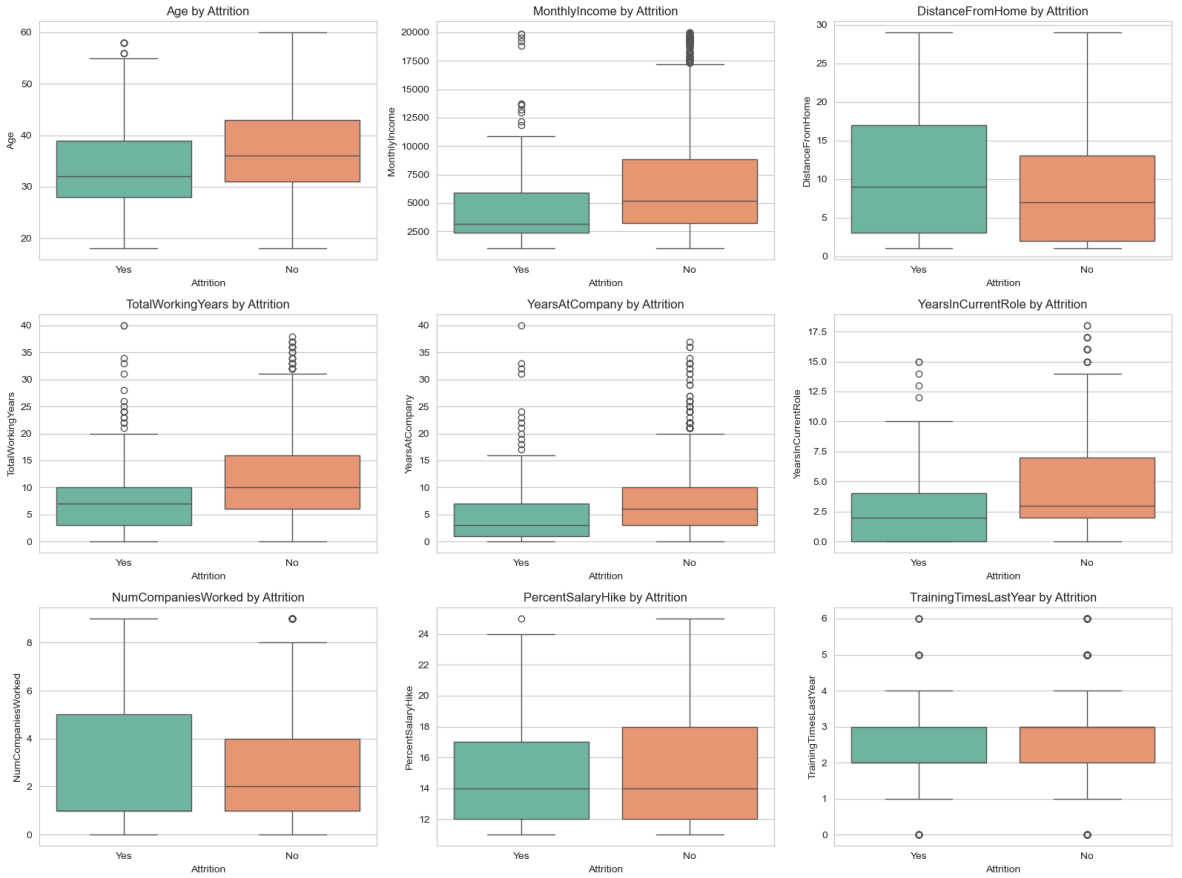
```
Dropped: ['EmployeeCount', 'Over18', 'StandardHours', 'EmployeeNumber'] |
New shape: (1470, 31)
```

In [10]:
```python
## 6.TARGET DISTRIBUTION

print(df["Attrition"].value_counts())
print((df["Attrition"].value_counts(normalize=True) * 100).round(2))

sns.countplot(data=df, x="Attrition", palette="Set2" , hue="Attrition")
plt.title("Attrition Class Distribution")
plt.show()
```

```
Attrition
No     1233
Yes     237
Name: count, dtype: int64
Attrition
No     83.88
Yes    16.12
Name: proportion, dtype: float64
```

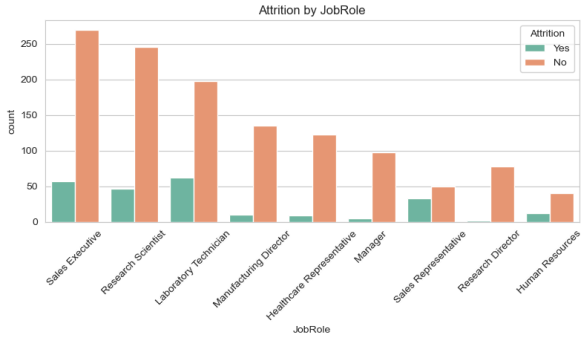
In [9]:
```python
## 7.NUMERIC FEATURES VS ATTRITION

numeric_features = ["Age", "MonthlyIncome", "DistanceFromHome", "TotalWor
                    "YearsAtCompany", "YearsInCurrentRole", "NumCompanie
                    "PercentSalaryHike", "TrainingTimesLastYear"]
numeric_features = [c for c in numeric_features if c in df.columns]
```

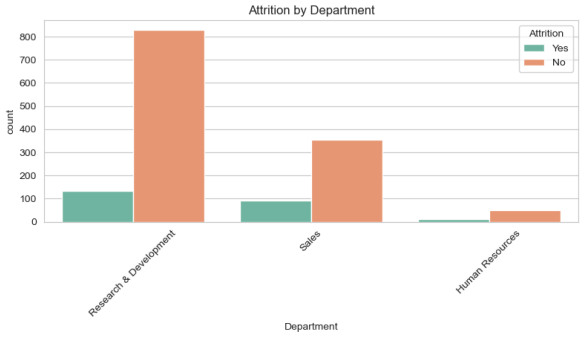
```
fig, axes = plt.subplots(3, 3, figsize=(16, 12))
axes = axes.flatten()
for i, col in enumerate(numeric_features):
    sns.boxplot(data=df, x="Attrition", y=col, ax=axes[i], palette="Set2"
    axes[i].set_title(f"{col} by Attrition")
plt.tight_layout()
plt.show()
```

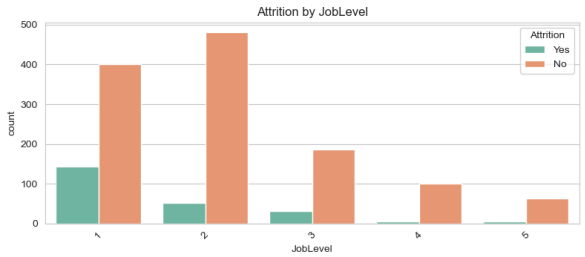


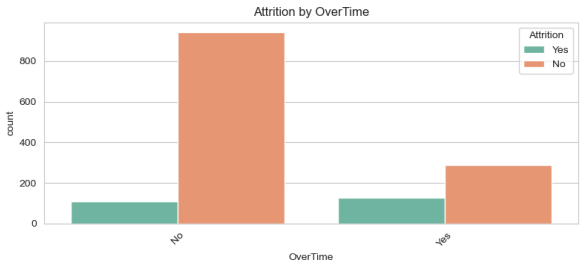
In [11]:

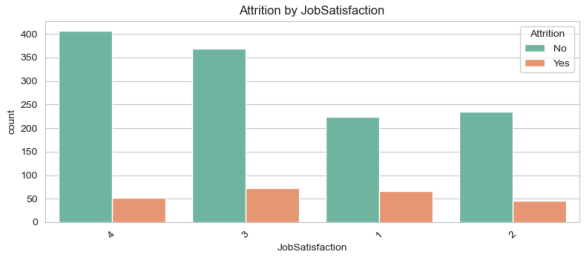
```
## 8.Categorical features vs Attrition

categorical_features = ["JobRole", "Department", "JobLevel", "OverTime",
                        "JobSatisfaction", "EnvironmentSatisfaction", "W
                        "MaritalStatus", "StockOptionLevel", "Performanc
categorical_features = [c for c in categorical_features if c in df.column

fig, axes = plt.subplots(5, 2, figsize=(16, 22))
axes = axes.flatten()
for i, col in enumerate(categorical_features):
    order = df[col].value_counts().index
    sns.countplot(data=df, x=col, hue="Attrition", ax=axes[i], order=orde
    axes[i].set_title(f"Attrition by {col}")
    axes[i].tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()
```

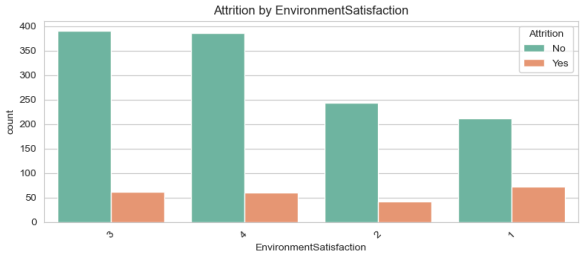


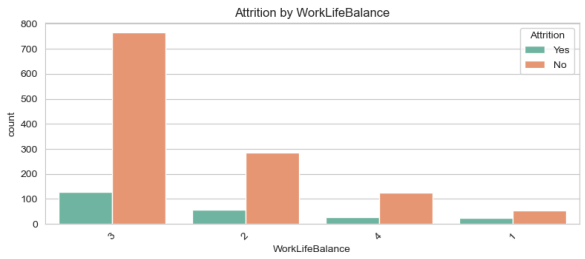


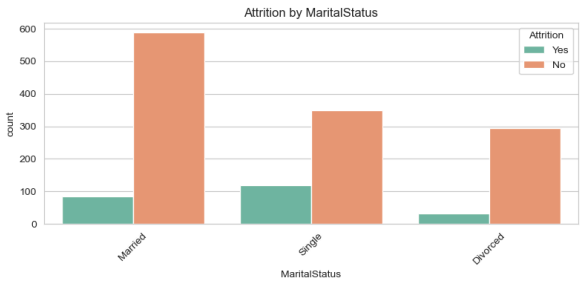


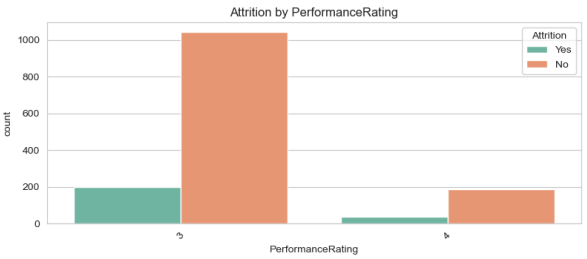




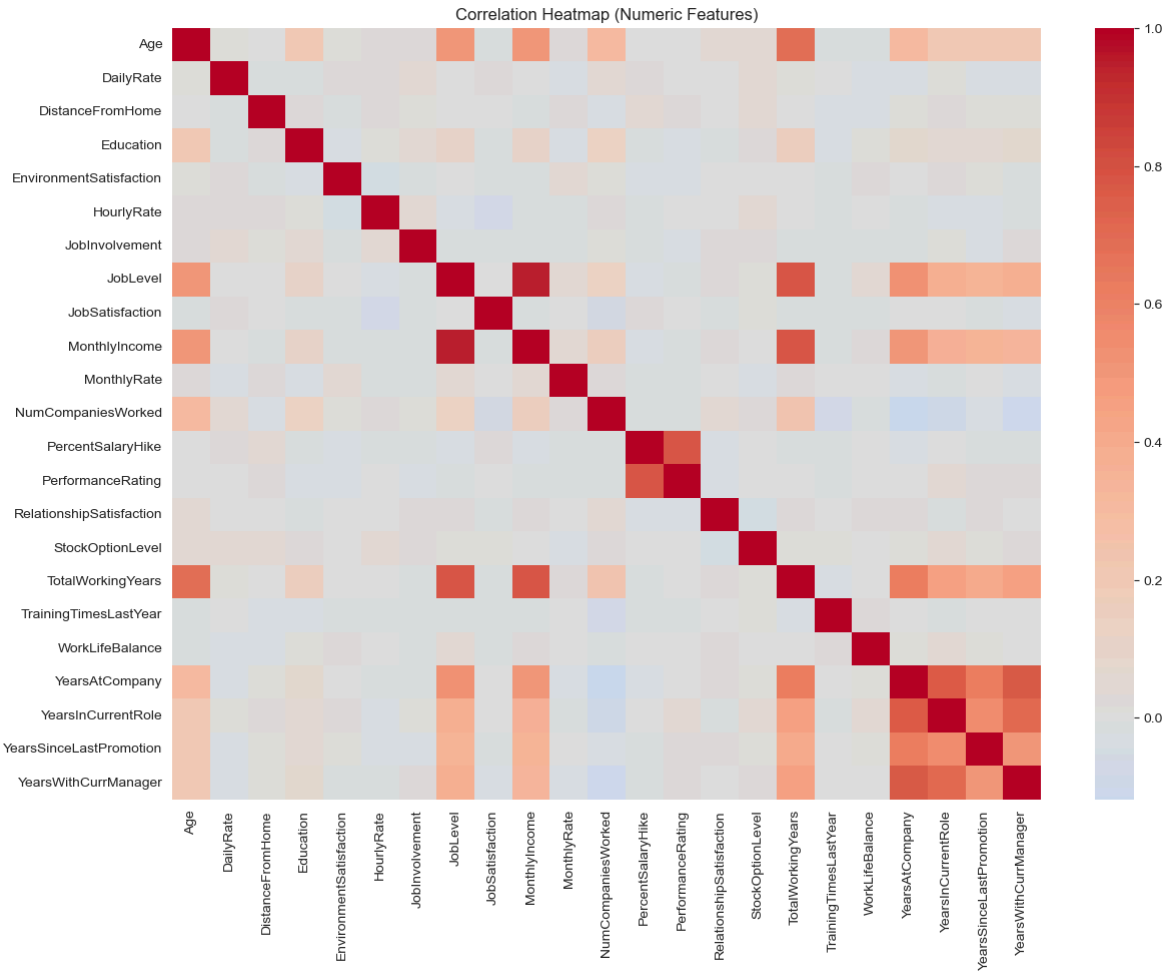








In [12]:
```
## 9.Correlation heatmap

plt.figure(figsize=(14, 10))
corr = df.select_dtypes(include=[np.number]).corr()
sns.heatmap(corr, cmap="coolwarm", center=0)
plt.title("Correlation Heatmap (Numeric Features)")
plt.show()
```



In [13]:
```
## 10.Encode features


df["Attrition_Flag"] = df["Attrition"].map({"Yes": 1, "No": 0})
categorical_cols = [c for c in df.select_dtypes(include="object").columns
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True
print("Encoded shape:", df_encoded.shape)
df_encoded.head()
```

```
Encoded shape: (1470, 45)
```

Out[13]:

	Age	DailyRate	DistanceFromHome	Education	EnvironmentSatisfaction	Hourly
0	41	1102	1	2	2	
1	49	279	8	1	3	
2	37	1373	2	2	4	
3	33	1392	3	4	4	
4	27	591	2	1	1	

In [14]:
```
## 11.Split & scale

X = df_encoded.drop(columns=["Attrition_Flag"])
y = df_encoded["Attrition_Flag"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
```

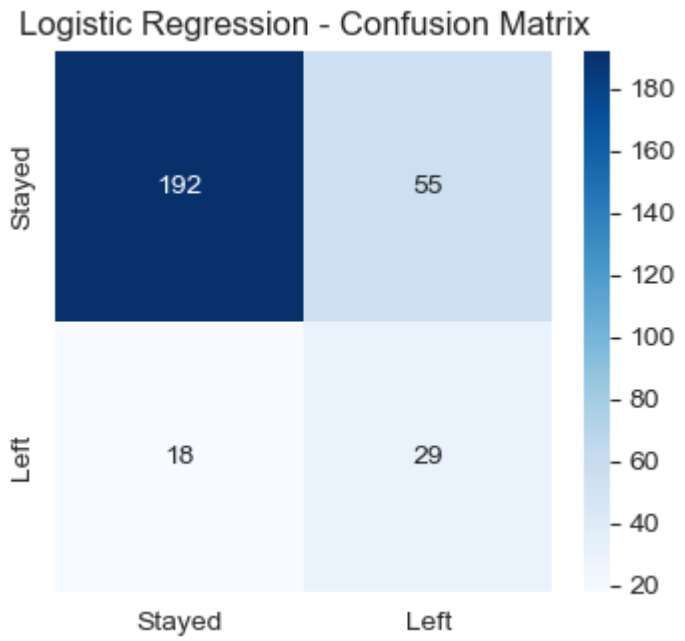
```
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [15]:
```
## 12.Train models

log_reg = LogisticRegression(max_iter=1000, class_weight="balanced", rand
log_reg.fit(X_train_scaled, y_train)

rf = RandomForestClassifier(n_estimators=300, max_depth=8, class_weight="
rf.fit(X_train, y_train)
print("Models trained.")
```

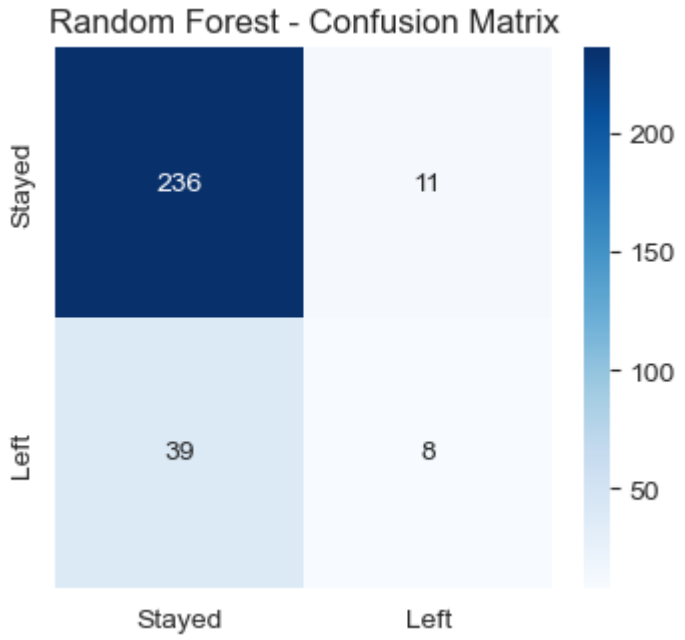
```
Models trained.
```

In [16]:
```
## 13.Evaluation function + results

def evaluate_model(name, y_true, y_pred, y_proba):
    print(f"--- {name} ---")
    print("Accuracy :", round(accuracy_score(y_true, y_pred), 3))
    print("Precision:", round(precision_score(y_true, y_pred), 3))
    print("Recall   :", round(recall_score(y_true, y_pred), 3))
    print("F1-score :", round(f1_score(y_true, y_pred), 3))
    print("ROC-AUC  :", round(roc_auc_score(y_true, y_proba), 3))
    print(classification_report(y_true, y_pred, target_names=["Stayed", "

    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(4, 3.5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=["Stayed", "Left"], yticklabels=["Stayed", "L
    plt.title(f"{name} - Confusion Matrix")
    plt.show()

y_pred_log = log_reg.predict(X_test_scaled)
y_proba_log = log_reg.predict_proba(X_test_scaled)[:, 1]
evaluate_model("Logistic Regression", y_test, y_pred_log, y_proba_log)

y_pred_rf = rf.predict(X_test)
y_proba_rf = rf.predict_proba(X_test)[:, 1]
evaluate_model("Random Forest", y_test, y_pred_rf, y_proba_rf)
```

```
--- Logistic Regression ---
Accuracy : 0.752
Precision: 0.345
Recall   : 0.617
F1-score : 0.443
ROC-AUC  : 0.798
              precision    recall  f1-score   support

      Stayed       0.91      0.78      0.84       247
        Left       0.35      0.62      0.44        47

    accuracy                           0.75       294
   macro avg       0.63      0.70      0.64       294
weighted avg       0.82      0.75      0.78       294
```

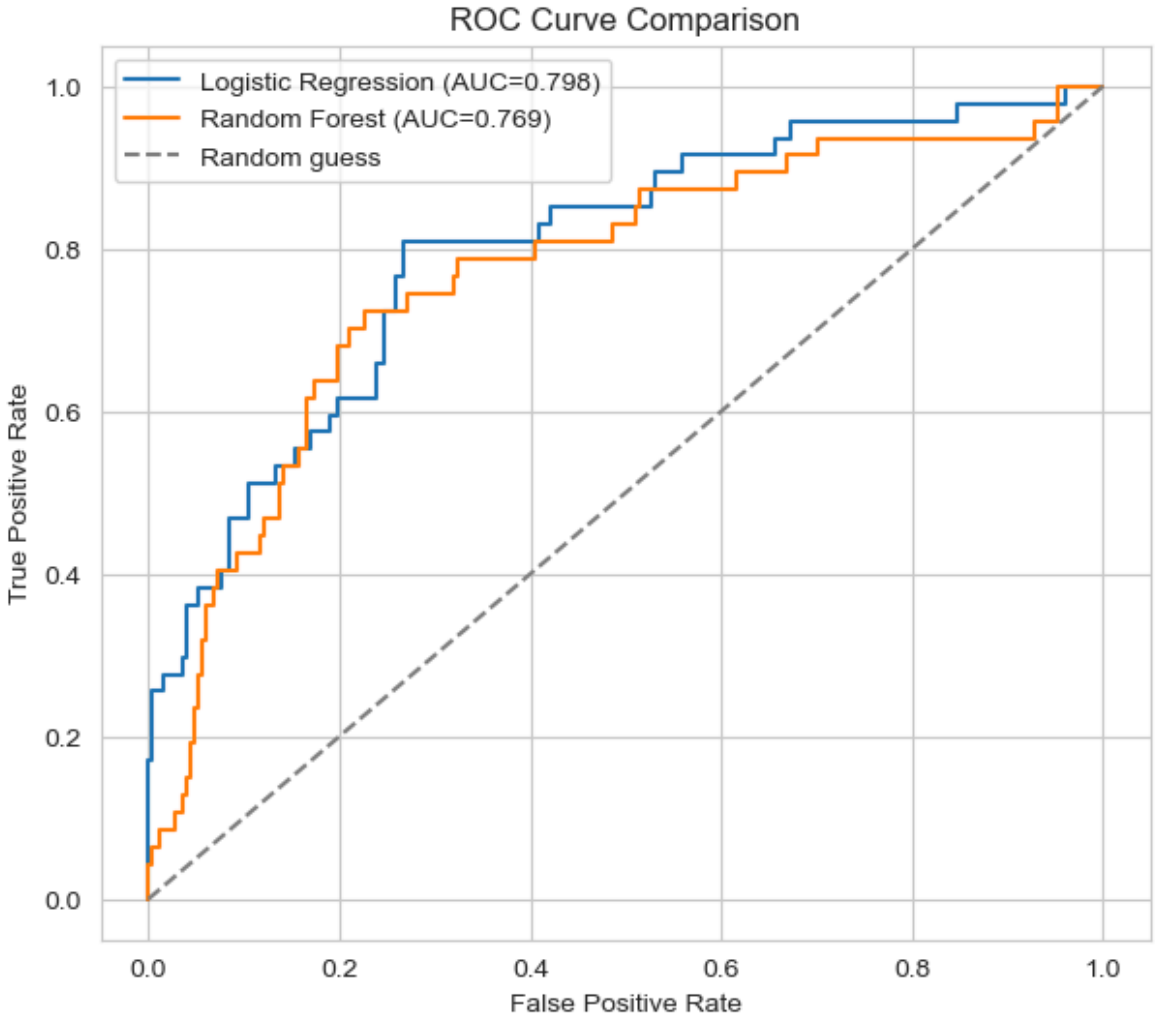



```
--- Random Forest ---
Accuracy : 0.83
Precision: 0.421
Recall   : 0.17
F1-score : 0.242
ROC-AUC  : 0.769
              precision    recall  f1-score   support

      Stayed       0.86      0.96      0.90       247
        Left       0.42      0.17      0.24        47

    accuracy                           0.83       294
   macro avg       0.64      0.56      0.57       294
weighted avg       0.79      0.83      0.80       294
```



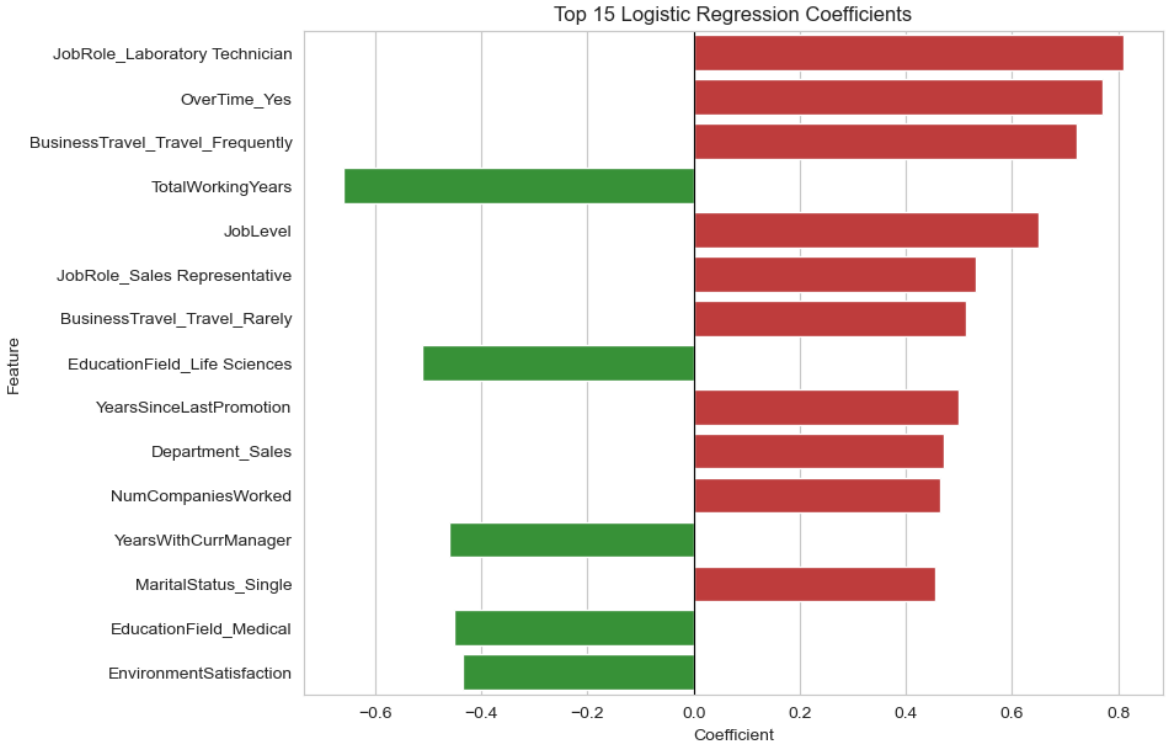
In [17]:
```
## 14.ROC comparison

fpr_log, tpr_log, _ = roc_curve(y_test, y_proba_log)
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_proba_rf)
```

```
plt.figure(figsize=(7, 6))
plt.plot(fpr_log, tpr_log, label=f"Logistic Regression (AUC={roc_auc_scor
plt.plot(fpr_rf, tpr_rf, label=f"Random Forest (AUC={roc_auc_score(y_test
plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Random gues
plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison"); plt.legend()
plt.show()
```

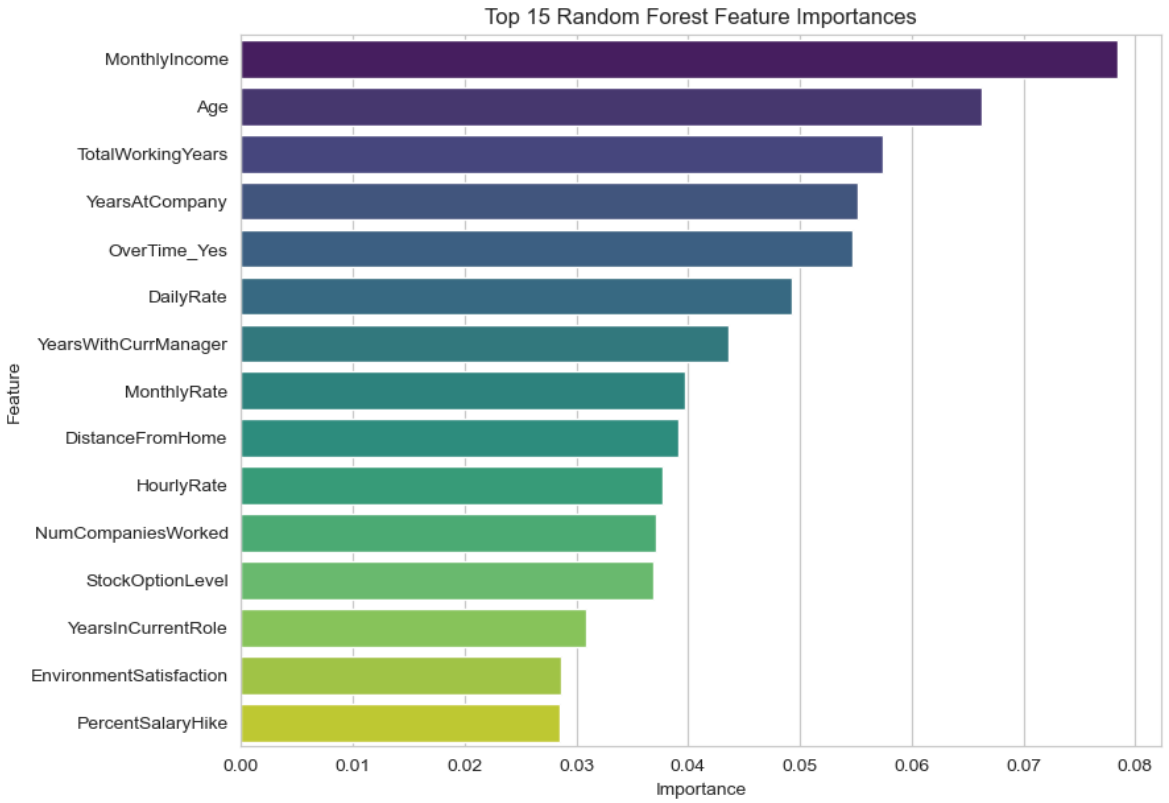


In [20]:
```
## 15.Explainability

coef_df = pd.DataFrame({"Feature": X.columns, "Coefficient": log_reg.coef
coef_df["AbsCoefficient"] = coef_df["Coefficient"].abs()
coef_df = coef_df.sort_values("AbsCoefficient", ascending=False).head(15)

plt.figure(figsize=(9, 7))
sns.barplot(data=coef_df, x="Coefficient", y="Feature",
            palette=["#d62728" if c > 0 else "#2ca02c" for c in coef_df["
plt.title("Top 15 Logistic Regression Coefficients")
plt.axvline(0, color="black", linewidth=0.8)
plt.show()

importance_df = pd.DataFrame({"Feature": X.columns, "Importance": rf.feat
importance_df = importance_df.sort_values("Importance", ascending=False).

plt.figure(figsize=(9, 7))
sns.barplot(data=importance_df, x="Importance", y="Feature", palette="vir
plt.title("Top 15 Random Forest Feature Importances")
plt.show()
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_2872\288894980.py:6: FutureWarnin
g:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for
the same effect.

  sns.barplot(data=coef_df, x="Coefficient", y="Feature",
```




```
C:\Users\HP\AppData\Local\Temp\ipykernel_2872\288894980.py:16: FutureWarni
ng:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for
the same effect.

  sns.barplot(data=importance_df, x="Importance", y="Feature", palette="vi
ridis")
```


Top 15 Random Forest Feature Importances
MonthlyIncome
Age
TotalWorkingYears
YearsAtCompany
OverTime_Yes
DailyRate
YearsWithCurrManager
MonthlyRate
DistanceFromHome
HourlyRate
NumCompaniesWorked
StockOptionLevel
YearsInCurrentRole
EnvironmentSatisfaction
PercentSalaryHike
Feature
0.00
0.01
0.02
0.03
0.04
0.05
0.06
0.07
0.08
Importance

In [ ]: